REAL-TIME MOTION CLASSIFICATION

with Wekinator & Processing

Complete Technical & Pedagogical Report

CLASS 1	CLASS 2	CLASS 3
STILL	**FLOW**	**CHAOS**
Dark Blue Tint	White Tint	Red Tint

Stack: Processing 4 ▪ Wekinator ▪ oscP5 ▪ netP5 ▪ kNN Classifier

Contents

1 Project Overview **3**
- 1.1 What the System Perceives . . . 3
- 1.2 Why This Architecture Is Interesting . . . 3

2 How the Three Parts Work Together **3**
- 2.1 End-to-End Data Flow . . . 4

3 VideoMotionDetection.pde — Full Explanation **5**
- 3.1 Libraries and What They Provide . . . 5
- 3.2 Global Variables — Purpose of Every Declaration . . . 5
- 3.3 The Motion Detection Algorithm — Line by Line . . . 6
  - 3.3.1 Why brightness and not RGB color? . . . 7
  - 3.3.2 Why threshold = 30? . . . 7
  - 3.3.3 Why multiply by 4? . . . 7
- 3.4 The Ring Buffer — How It Works . . . 7
- 3.5 The OSC Send Function . . . 8
- 3.6 The On-Screen Displays . . . 8
  - 3.6.1 HUD panel (top left) . . . 8
  - 3.6.2 Buffer waveform (bottom left) . . . 8

4 Why the Number of Inputs Is 30 (and Not Any Other Number) **9**
- 4.1 The Naive Approach and Why It Fails . . . 9
- 4.2 What 30 Values Add: A Temporal Portrait . . . 9
- 4.3 Why 30 Specifically? . . . 10
- 4.4 The Critical Constraint: bufferSize Must Match Wekinator's Input Count . . . 10

5 CameraTint.pde — Full Explanation **11**
- 5.1 State Constants and Initial State . . . 11
- 5.2 The Tint Color Mapping . . . 11
- 5.3 The Lerp Smoothing System . . . 12
- 5.4 OSC Reception — Every Guard and Why It Exists . . . 12

6 Why the Number of Outputs Is 1 (and Not 3) **13**
- 6.1 Two Completely Different Ways to Encode a Class . . . 13
  - 6.1.1 Method 1 — One-hot encoding (3 outputs) . . . 13
  - 6.1.2 Method 2 — Class index encoding (1 output) . . . 13
- 6.2 Why 3 Outputs Would Break This System . . . 13
- 6.3 Why the Intuition "3 Classes = 3 Outputs" Is Wrong Here . . . 14
- 6.4 Why the Output Must Use Classification Mode (Not Regression) . . . 14
- 6.5 Summary: The 1-Output Rule . . . 15

7 Wekinator Configuration — Exact Setup **15**
- 7.1 The Wekinator Algorithm: k-Nearest Neighbors . . . 15

8 The Three Motion Classes — Complete Specification **16**
- 8.1 Class 1: STILL . . . 16
- 8.2 Class 2: FLOW . . . 16

8.3 Class 3: CHAOS . . . 17

9 Training Procedure — Step by Step **17**
9.1 Pre-Training Checklist . . . 17
9.2 Launch Sequence . . . 18
9.3 Recording Class 1: STILL . . . 18
9.4 Recording Class 2: FLOW . . . 18
9.5 Recording Class 3: CHAOS . . . 19
9.6 Training and Running the Model . . . 19
9.7 Iterative Improvement . . . 19

10 OSC Communication — Complete Reference **20**
10.1 OSC Message Format . . . 20
10.2 Port Map . . . 20
10.3 Debugging OSC Connectivity . . . 20

11 Improving Classification Quality **20**
11.1 Data Quality Principles . . . 21
11.2 Parameter Tuning . . . 21

12 Educational Significance **21**
12.1 Limitations . . . 22
12.2 Possible Extensions . . . 22

13 Quick Reference Card **23**

1. Project Overview

This project implements a complete, real-time interactive machine learning pipeline using a webcam, two Processing sketches, and Wekinator. The webcam feed is analyzed frame by frame to measure how much motion is occurring. That measurement is packaged into a numerical description and sent to Wekinator, which uses machine learning to decide which of three categories the motion belongs to. The predicted category is then sent back to a second Processing sketch, which changes the color tint of the camera display.

At a Glance — Key Numbers

Webcam frames compared per second	≈ 30
Motion features per OSC message	**30** (temporal ring buffer)
OSC input port (Wekinator)	6448
OSC output port (CameraTint)	12000
Number of classifier outputs	**1**
Classifier output values	1 (STILL) / 2 (FLOW) / 3 (CHAOS)
Color lerp smoothing factor	0.1 per draw() frame
Pixel brightness threshold	30 out of 255
Motion amplification factor	$\times 4$

1.1 What the System Perceives

The system perceives exactly one thing: the aggregate intensity of frame-to-frame brightness change across the entire camera frame. It cannot identify objects, faces, body pose, or perform any form of scene understanding. It sees motion as a single dimensionless quantity. This constraint is not a weakness — it is a deliberate design choice that makes the pipeline simple enough to understand completely yet rich enough to demonstrate genuine machine learning.

1.2 Why This Architecture Is Interesting

The standard approach to motion-reactive applications is to hard-code thresholds: *if motion is above 0.7, trigger event A; if below 0.1, trigger event B*. This project does the opposite. You provide examples and Wekinator discovers the boundaries between classes from those examples. The categories STILL, FLOW, and CHAOS are meaningful because you define them by demonstration, not because a programmer wrote explicit rules.

This is the central insight of interactive machine learning: **meaning can emerge from demonstration rather than from explicit rule writing.**

2. How the Three Parts Work Together

The project consists of three concurrent processes that communicate exclusively through OSC (Open Sound Control), a network protocol transmitted over the local loopback interface (`127.0.0.1`). They share no memory and no files.

```
                VideoMotionDetection.pde
↓   /wek/inputs    30 float values    → localhost:6448
                      Wekinator
↓   /wek/outputs   1 integer value    → localhost:12000
                    CameraTint.pde
```

Component	Role	Sends to	Listens on
VideoMotionDetection	Input generator	port 6448	port 12001
Wekinator	ML classifier	port 12000	port 6448
CameraTint	Output visualizer	—	port 12000

All three must be running simultaneously. The launch sequence is covered in Section 9.

2.1 End-to-End Data Flow

Each step below represents one stage in the pipeline, from raw photons to rendered color on screen.

1. **Webcam capture.** The webcam captures live video frames continuously at approximately 30 fps. Each new frame triggers `captureEvent()`.
2. **Frame comparison.** The current frame pixels are compared against `prevFrame` pixel by pixel. Brightness delta is computed per pixel.
3. **Threshold filter.** Pixels whose brightness delta exceeds 30 are counted as motion pixels (`whiteCount`).
4. **Normalization.** `motionAmount = (whiteCount / sampledCount) × 4`. This produces a float with an amplified dynamic range.
5. **Buffer shift.** The 30-slot ring buffer shifts left by one slot. The newest `motionAmount` fills slot 29.
6. **OSC send.** All 30 buffer values are packed into a single `/wek/inputs` message and transmitted to port `6448`.
7. **Classification.** Wekinator receives the 30-feature vector, runs the trained kNN classifier, and produces a class integer.
8. **OSC receive.** `CameraTint.pde` listens on port `12000` for `/wek/outputs` carrying the predicted integer.
9. **Tint update.** The integer (1, 2, or 3) maps to a target color. `lerpColor()` smoothly interpolates toward that color each frame.

Important

Steps 1 through 6 execute on every single `draw()` frame. Wekinator processes incoming OSC messages asynchronously and sends output predictions continuously while in Run mode. The entire round-trip latency is typically under one frame at 30 fps, making the visual response feel immediate.

3. VideoMotionDetection.pde — Full Explanation

This sketch is the engine of the project. It accesses the webcam, compares each new frame against the previous one, computes a motion intensity value, stores a history of recent values in a ring buffer, and sends that buffer to Wekinator as a feature vector every frame.

3.1 Libraries and What They Provide

```
import processing.video.*;   // Capture class -- reads webcam frames
import oscP5.*;              // OscP5 -- creates OSC socket, builds messages
import netP5.*;              // NetAddress -- stores destination host:port
```

Listing 1: Library imports

- `processing.video.*` provides the `Capture` class which interfaces with physical camera devices through the operating system's video API.
- `oscP5.*` provides `OscP5` for creating a local UDP socket and `OscMessage` for constructing typed OSC packets.
- `netP5.*` provides `NetAddress`, which encapsulates a destination hostname and port into a single reusable object.

3.2 Global Variables — Purpose of Every Declaration

```
Capture cam;
PImage prevFrame;

OscP5 oscP5;
NetAddress wekinator;

String wekHost = "127.0.0.1";
int wekPort = 6448;
String oscAddress = "/wek/inputs";

int localPort = 12001;

float threshold = 30;
```

```
float motionAmount = 0;

int bufferSize = 30;
float[] motionBuffer = new float[bufferSize];

boolean mirror = true;
```

Listing 2: Global state declarations

Variable	Purpose and reasoning
`cam`	The webcam object. `cameras[0]` selects the first available camera detected by the OS. If you have multiple cameras, increment this index.
`prevFrame`	A copy of the previous frame used for difference computation. On the very first draw() call it is `null`, so the sketch returns early and waits for a second frame before comparing.
`oscP5`	The local OSC endpoint initialized on port 12001. oscP5 requires a listening port even when only sending outbound messages. Port 12001 is chosen to avoid conflict with CameraTint (12000).
`wekinator`	Destination address object pointing to `127.0.0.1:6448`. Wekinator's input port.
`threshold`	Minimum brightness delta for a pixel to count as changed. Value of 30 filters camera sensor noise and JPEG artifacts while responding to real movement. Explained fully in Section 3.3.1.
`motionAmount`	Current normalized motion intensity, computed every frame from the ratio of changed pixels to total pixels, amplified by $\times 4$.
`bufferSize`	Length of the temporal ring buffer and therefore the exact number of inputs Wekinator receives. Must always match Wekinator's configured input count. Full explanation in Section 4.
`motionBuffer`	The ring buffer array. Index 0 is the oldest value. Index 29 is the most recent.
`mirror`	Horizontally flips the camera display only. Does not affect the pixel comparison, because both `cam.pixels` and `prevFrame.pixels` are read before the mirror transform.

3.3 The Motion Detection Algorithm — Line by Line

```
int whiteCount = 0;
int sampledCount = 0;

for (int y = 0; y < cam.height; y++) {
  for (int x = 0; x < cam.width; x++) {
    int index = x + y * cam.width;    // 2D coords -> 1D array index

    color c1 = cam.pixels[index];        // current frame pixel
    color c2 = prevFrame.pixels[index]; // same pixel, previous frame

```

```
    float b1 = brightness(c1);    // HSB brightness channel, 0-255
    float b2 = brightness(c2);

    float diff = abs(b1 - b2);    // absolute brightness difference

    if (diff > threshold) {       // threshold = 30
      whiteCount++;               // pixel counts as motion
    }
    sampledCount++;               // always increment total
  }
}

motionAmount = ((float)whiteCount / (float)sampledCount) * 4;
```

Listing 3: Per-pixel brightness comparison in draw()

3.3.1 Why brightness and not RGB color?

Comparing the HSB brightness channel rather than raw RGB values makes the detector robust to color temperature shifts. If the room lighting changes slightly, the red, green, and blue channel values of every pixel may shift substantially, producing widespread false motion detections. Brightness, which represents overall luminance rather than hue, remains far more stable under minor lighting variations.

3.3.2 Why threshold = 30?

Camera sensors produce noise — tiny random fluctuations in pixel values even when the scene contains no motion. JPEG encoding also introduces block-level compression artifacts. A threshold of 30 out of 255 (about 12%) ignores this sub-perceptual noise while still responding to genuine movement.

Threshold value	Effect
10	Too sensitive. Sensor noise and lighting flicker register as motion.
30	Default. Filters noise, responds to real movement.
50	Less sensitive. Only large or fast movements register.
80+	Very insensitive. Subtle movement in FLOW class may not register.

3.3.3 Why multiply by 4?

In a typical scene, even vigorous motion affects only a fraction of all pixels between consecutive frames. Without amplification, the `motionAmount` during intense movement might reach only 0.15–0.20, making the full range of the value very compressed. The $\times 4$ multiplier stretches this into a more useful dynamic range, making the waveform visualizer more readable and giving Wekinator better numerical contrast between motion intensities when comparing training examples.

3.4 The Ring Buffer — How It Works

```
void updateMotionBuffer(float value) {
  for (int i = 0; i < bufferSize - 1; i++) {
```

```
    motionBuffer[i] = motionBuffer[i + 1]; // shift every slot left by one
  }
  motionBuffer[bufferSize - 1] = value;     // insert newest at the end
}
```

Listing 4: updateMotionBuffer() – ring buffer shift

Every frame, all 30 slots shift one position toward index 0, and the newest `motionAmount` is placed at index 29. The buffer always contains the last 30 frames of motion history. The full reasoning for why this temporal window is the most important design decision in the project is explained in detail in Section 4.

3.5 The OSC Send Function

```
void sendBufferToWekinator() {
  OscMessage msg = new OscMessage("/wek/inputs"); // Wekinator's expected address

  for (int i = 0; i < bufferSize; i++) {
    msg.add(motionBuffer[i]);    // append each of the 30 floats
  }

  oscP5.send(msg, wekinator);  // transmit to 127.0.0.1:6448
}
```

Listing 5: sendBufferToWekinator() – packaging and transmitting features

This function is called on every `draw()` frame. Wekinator receives approximately 30 messages per second, each containing the complete 30-value motion history. Wekinator classifies each message independently and immediately returns a prediction — it does not accumulate messages over time.

3.6 The On-Screen Displays

3.6.1 HUD panel (top left)

Shows the OSC destination address, the active OSC address string, the live `motionAmount` value at four decimal places, and the number of features being sent. If the host/port shown here does not match Wekinator's configured input port, no messages are reaching Wekinator.

3.6.2 Buffer waveform (bottom left)

```
beginShape();
for (int i = 0; i < bufferSize; i++) {
  float x = map(i, 0, bufferSize - 1, gx, gx + gw);
  float y = map(motionBuffer[i], 0, 1, gy + gh, gy);
  vertex(x, y);
}
endShape();
```

Listing 6: drawBufferPreview() – waveform visualization

This renders the 30 buffer slots as a green line graph. It is the most important diagnostic tool in the entire project. Before recording any training examples, watch this waveform and verify that each motion type produces a visually distinct shape:

- **STILL** → nearly flat line close to zero
- **FLOW** → consistent moderate-height wave, smooth contour
- **CHAOS** → high-amplitude jagged spikes, irregular profile

If the three shapes look similar on the waveform, Wekinator cannot distinguish them regardless of how many examples you record.

4. Why the Number of Inputs Is 30 (and Not Any Other Number)

This is the most important design decision in the project. Understanding it fully will clarify what Wekinator is actually learning, why the buffer exists, and why changing it requires changing Wekinator too.

4.1 The Naive Approach and Why It Fails

The simplest possible feature would be to send a single float — the current `motionAmount` — to Wekinator as one input. Wekinator would then try to classify motion based on one number per message.

This fails because the three target classes cannot be distinguished by a single instantaneous value:

- STILL produces `motionAmount` values near zero.
- FLOW produces values roughly between 0.10 and 0.50.
- CHAOS produces values above 0.40, sometimes spiking above 1.0.

The ranges of FLOW and CHAOS overlap substantially. A brief pause mid-gesture produces a value that looks like STILL for one frame. The start and end of any motion produces FLOW-range values regardless of the overall motion class. A single number carries no information about whether the current value is typical or anomalous for the behavior happening right now.

4.2 What 30 Values Add: A Temporal Portrait

By keeping 30 consecutive `motionAmount` values and sending all of them together as one feature vector, the input stops being a snapshot and becomes a short portrait of recent motion history.

Consider how each class appears across 30 values:

STILL — 30 values all near zero, consistent, no sudden changes:

$$[0.01,\ 0.00,\ 0.02,\ 0.01,\ 0.00,\ 0.01,\ 0.02,\ 0.00,\ \ldots,\ 0.01]$$

FLOW — 30 values moderate, consistent, slowly varying:

$$[0.21,\ 0.24,\ 0.27,\ 0.29,\ 0.28,\ 0.26,\ 0.23,\ 0.25,\ \ldots,\ 0.26]$$

CHAOS — 30 values high on average, wildly irregular, large jumps between consecutive slots:

$$[0.72,\ 0.18,\ 0.91,\ 0.04,\ 0.85,\ 0.31,\ 0.78,\ 0.05,\ \ldots,\ 0.95]$$

These three patterns are radically different from each other in 30-dimensional feature space. Even a simple kNN classifier separates them reliably because:

- The Euclidean distance between any STILL vector and any CHAOS vector is enormous.
- A brief anomalous frame (e.g., a cough during STILL) affects only one slot out of 30 instead of dominating the entire feature.
- The temporal texture of CHAOS — the pattern of large irregular jumps between consecutive slots — is visible in the vector as consecutive elements that differ dramatically.

4.3 Why 30 Specifically?

30 corresponds to approximately one second of camera frames at 30 fps. This duration is long enough to capture the temporal signature of each motion quality while being short enough that classification responds quickly to behavioral changes.

Buffer size	Problem
5 frames ($\approx$ 0.17s)	Window too short. Temporal patterns unreliable. Transient noise dominates.
15 frames ($\approx$ 0.5s)	Better, but the transition from CHAOS to STILL takes only 0.5s to flush.
30 frames ($\approx$ 1.0s)	Good balance: captures temporal texture, transitions within $\approx$ 1 second.
60 frames ($\approx$ 2.0s)	Sluggish. Switching from CHAOS to STILL takes 2 full seconds to reclassify.
120 frames ($\approx$ 4.0s)	Too slow. Motion history dominates over current behavior entirely.

4.4 The Critical Constraint: bufferSize Must Match Wekinator's Input Count

Critical: These two values must always be equal

`bufferSize = 30` (in VideoMotionDetection.pde)

$\Updownarrow$

Number of Inputs = 30 (in Wekinator project settings)

Wekinator is configured to receive exactly 30 inputs per message because the sketch always sends exactly 30 float arguments. If these values differ:

- **bufferSize $<$ Wekinator inputs:** Wekinator receives fewer values than expected. It shows an OSC mismatch warning and refuses to process the message. Training appears to proceed but classification fails silently.

- **bufferSize > Wekinator inputs:** Wekinator truncates the incoming message to the number it expects. The temporal window the model learns from is different from the one used at runtime, producing inconsistent and unreliable behavior.

The input count of 30 in Wekinator is not a preference. It is the exact cardinality of the array that `sendBufferToWekinator()` transmits. They must always be equal. If you change `bufferSize` in the sketch, you must open Wekinator, delete the project, and recreate it with the new input count.

5. CameraTint.pde — Full Explanation

This sketch is the visible output layer. It transforms an abstract integer prediction from Wekinator into a color experience. Its design prioritizes smooth, organic transitions.

5.1 State Constants and Initial State

```
final int STILL = 1;    // matches Wekinator class label 1
final int FLOW  = 2;    // matches Wekinator class label 2
final int CHAOS = 3;    // matches Wekinator class label 3

int currentState = STILL; // safe default before OSC arrives
```

Listing 7: Class label constants

Using named constants rather than raw integers throughout the code prevents bugs where a magic number like 2 could be confused for a screen coordinate or loop index. Starting with `currentState = STILL` means the display shows a defined, neutral state before Wekinator sends its first prediction, rather than defaulting to zero (which is not a valid class label).

5.2 The Tint Color Mapping

```
void updateTintByState() {
  if (currentState == STILL)
    targetTint = color(90);            // dark desaturated blue-grey
  else if (currentState == FLOW)
    targetTint = color(255);           // pure neutral white
  else if (currentState == CHAOS)
    targetTint = color(255, 80, 80);   // saturated red
}
```

Listing 8: updateTintByState() – mapping states to colors

The three colors carry perceptual associations that match the motion classes:

State	Color call	Perceptual meaning
STILL	`color(90)`	Cold, dark, reduced luminosity. Stillness, dormancy.
FLOW	`color(255)`	Neutral, clear, natural. The camera appears unfiltered.
CHAOS	`color(255,80,80)`	High-energy red. Alarm, urgency, intensity.

A semi-transparent rectangle is also drawn over the entire frame:

```
fill(red(currentTint), green(currentTint), blue(currentTint),
    80);
rect(0, 0, width, height);
```

Listing 9: Color wash overlay for darkened regions

This creates a color wash visible even in the darkest regions of the camera frame, where the `tint()` function alone might produce no perceptible change.

5.3 The Lerp Smoothing System

```
currentTint = lerpColor(currentTint, targetTint, 0.1);
```

Listing 10: Smooth color interpolation in draw()

`lerpColor()` interpolates each RGB channel independently. At a factor of 0.1, it moves 10% of the remaining distance toward the target color on every frame. This is exponential smoothing — fast at first, slowing as it approaches the target.

Why this is essential: Wekinator's classifier can produce brief incorrect predictions at the moment of transition between classes. Without smoothing, each wrong frame would produce an instantaneous flash of the wrong color. With a lerp factor of 0.1, a single erroneous frame moves the display only 10% toward the wrong color before the correct prediction restores the direction of change. The visual effect is a smooth, believable transition that absorbs transient misclassifications gracefully.

Lerp factor	Effect
0.03	Very smooth, but sluggish. Transitions take 2–3 seconds to complete.
0.1	Default. Good balance between smoothness and responsiveness at 30 fps.
0.3	Snappier, but occasional transient misclassifications become visible.
1.0	No smoothing. Every frame jumps directly to the target color.

5.4 OSC Reception — Every Guard and Why It Exists

```
void oscEvent(OscMessage msg) {
  if (msg.checkAddrPattern("/wek/outputs")) {

    if (msg.typetag().length() == 1) {
      int newState = round(msg.get(0).floatValue());
      currentState = newState;
    }

    println("State: " + currentState);
  }
}
```

Listing 11: oscEvent() – receiving predictions from Wekinator

- `checkAddrPattern("/wek/outputs")` ignores any OSC message that does not carry Wekinator's output address. This prevents other OSC traffic on the local network from corrupting the state.

- `typetag().length() == 1` confirms that exactly one argument was received. This guards against Wekinator being misconfigured with more than one output (see Section 6 for why exactly one output is correct).
- `round(msg.get(0).floatValue())` handles the fact that Wekinator transmits all values as floats over OSC. The class label arrives as `1.0f`, `2.0f`, or `3.0f`. Rounding is safer than casting: plain integer casting would truncate 1.87 to 1, which could cause incorrect state assignment near class boundaries. Rounding gives 2, which is the correct interpretation.

6. Why the Number of Outputs Is 1 (and Not 3)

This is the second most common source of confusion in the project, after the input count. The intuition that leads to the wrong answer is: *"There are three classes, so there should be three outputs."* This section explains completely why that intuition is wrong and why exactly one output is correct.

6.1 Two Completely Different Ways to Encode a Class

There are two fundamentally different ways to represent a multi-class prediction numerically. They are not interchangeable.

6.1.1 Method 1 — One-hot encoding (3 outputs)

In one-hot encoding, each class gets its own output channel. The output that is active (set to 1) identifies the predicted class:

Predicted class	Output 1	Output 2	Output 3
STILL	1	0	0
FLOW	0	1	0
CHAOS	0	0	1

This is common in neural networks and softmax classifiers, where each output represents the probability of its class.

6.1.2 Method 2 — Class index encoding (1 output)

In class index encoding, a single output channel carries an integer that identifies the class:

Predicted class	Output 1
STILL	1
FLOW	2
CHAOS	3

This is what Wekinator uses by default in classification mode. The three categories are encoded as three distinct integer values on one channel.

6.2 Why 3 Outputs Would Break This System

If Wekinator were configured with 3 outputs in classification mode, it would try to predict a separate class for each output channel independently. This creates two serious problems.

Problem 1: The OSC message would have 3 arguments instead of 1.

The guard in `oscEvent()` checks `typetag().length() == 1`. With 3 outputs, Wekinator sends a message with 3 float arguments and a typetag of `"fff"`, length 3. The guard fails. `currentState` is never updated. The tint never changes.

Problem 2: Each output would be classified as its own binary problem.

Wekinator would try to classify output 1 as {1 or something else}, output 2 as {1 or something else}, and output 3 as {1 or something else}, independently. This is not a meaningful formulation of the three-class problem and would require a completely different training and interpretation strategy that `CameraTint.pde` does not implement.

6.3 Why the Intuition "3 Classes = 3 Outputs" Is Wrong Here

The number of outputs is determined by the **structure of the prediction task**, not by the number of classes.

- If you need to predict **one class from a list of N mutually exclusive options**, you need **1 output** that carries an integer in the range $[1, N]$.
- If you need to predict **N independent continuous control values** simultaneously (e.g., the X and Y position of a cursor, plus a zoom level), you need **N outputs**.
- If you need to predict **N probabilities** (one per class, summing to 1), you need **N outputs** and a softmax output layer — which Wekinator in standard classification mode does not use.

This project predicts one class from three mutually exclusive options. The correct output count is therefore 1.

6.4 Why the Output Must Use Classification Mode (Not Regression)

Wekinator offers two output modes.

Mode	Output values	When to use
Regression	Continuous floats: 1.73, 2.44	Controlling a fader, mapping to brightness
Classification	Discrete integers: 1, 2, 3	Predicting one of N mutually exclusive classes

Using regression would produce intermediate values like 1.5 or 2.3 that are meaningless in this context. The `round()` call in `oscEvent()` would push 1.5 to either 1 or 2 depending on rounding direction, and the classification would become unstable near the boundaries between classes. Classification mode snaps predictions to whole integers (1, 2, or 3), which is exactly what the sketch expects.

6.5 Summary: The 1-Output Rule

Why exactly 1 output

1. Three classes, one prediction at a time → 1 output carrying the class index.
2. `CameraTint.pde` reads `msg.get(0)` (the first argument). If there is no first argument, or there are multiple, the system breaks.
3. The typetag guard `typetag().length() == 1` will silently block all messages if Wekinator is configured with 3 outputs.
4. Classification mode snaps to integers; regression mode produces unstable fractional values between classes.

7. Wekinator Configuration — Exact Setup

Wekinator must be configured precisely to match the expectations of both Processing sketches. Any mismatch in port numbers, input counts, or output types will cause the system to fail silently.

Required Wekinator Project Settings

OSC Input Port	6448
OSC Input Address	`/wek/inputs`
Number of Inputs	**30** (must match `bufferSize`)
Number of Outputs	**1**
Output Type	**Classification** (not regression)
OSC Output Host	`localhost` / `127.0.0.1`
OSC Output Port	12000
OSC Output Address	`/wek/outputs`

7.1 The Wekinator Algorithm: k-Nearest Neighbors

When using classification mode, Wekinator defaults to a k-Nearest Neighbors (kNN) classifier with $k = 3$. kNN classifies a new input by finding the three closest training examples in feature space and taking a majority vote of their labels.

This is well suited to this project because:

- kNN requires no explicit optimization phase — the model is the training data itself. Training is instantaneous.
- It handles non-linear class boundaries naturally.
- It is sensitive to training data density, meaning more examples give better coverage of the feature space.
- It degrades gracefully with ambiguous data rather than crashing or producing extreme values.

Distance is computed across all 30 temporal buffer slots simultaneously. Two motion patterns

that feel qualitatively similar but have different temporal shapes will be separated in 30-dimensional space if their slot-by-slot values differ.

8. The Three Motion Classes — Complete Specification

Each class must produce a clearly distinct pattern in the 30-slot motion buffer. Before recording any training examples, watch the buffer waveform in VideoMotionDetection and verify that your three motion types look visually different. If they do not look different on the waveform, Wekinator cannot distinguish them.

8.1 Class 1: STILL

CLASS 1 — STILL

Wekinator label 1
Tint color `color(90)` — dark desaturated blue-grey
Target motionAmount below 0.05 consistently
Buffer appearance Flat line near zero with tiny noise spikes
Definition Near-complete absence of intentional motion. Only residual sensor noise, minor environmental changes (dust, air currents, background lighting flicker), or very subtle involuntary movement.
Demonstration Stand completely still. Minimize breathing movement. Place a static object in front of a motionless background. Even chest movement during breathing can register if the subject fills the frame.
Training strategy Record at least 5 separate runs of 4–6 seconds each. Vary your position between runs so the model sees STILL from different parts of the frame. If values exceed 0.10 during recording, something in the background is moving and should be eliminated.
Common mistake Recording STILL while shifting weight between feet or looking around. These register as brief FLOW pulses and contaminate the training data.

8.2 Class 2: FLOW

CLASS 2 — FLOW

Wekinator label 2
Tint color `color(255)` — pure neutral white
Target motionAmount 0.10–0.40 consistently, without spikes or pauses
Buffer appearance Consistent moderate-height wave, smooth contour
Definition Continuous, sustained, moderate-intensity motion. The defining quality is *continuity over time*, not a particular speed. FLOW never stops and never bursts.
Demonstration Slowly waving both arms in a figure-eight pattern. Walking in front of the camera at a steady pace. Swaying the body gently from side to side. Drawing large slow circles in the air. The motion must be ongoing without pauses.

Training strategy Record 5–8 runs of at least 5 seconds each. Vary the body part, speed, and type of motion across runs. All should still register as FLOW if they share the quality of sustained continuity. Keep the motion going without pausing.

Common mistake Moving too fast (waveform spikes like CHAOS) or stopping briefly between waves (waveform drops like STILL). Both produce ambiguous training data.

8.3 Class 3: CHAOS

CLASS 3 — CHAOS

Wekinator label 3

Tint color `color(255, 80, 80)` — saturated red

Target motionAmount regularly exceeding 0.40, with spikes above 0.80

Buffer appearance High-amplitude jagged spikes, large jumps between consecutive slots, no predictable shape

Definition High-intensity, irregular, unpredictable motion. CHAOS is defined by irregularity rather than raw intensity alone. Rhythmic fast motion can drift toward FLOW if it becomes too predictable.

Demonstration Shaking both hands rapidly and changing direction unpredictably. Sudden large jerky arm movements. Running in place. Waving a cloth aggressively in front of the camera. The key quality is unpredictability — each frame should differ substantially from the last.

Training strategy Record 5–8 runs. Break patterns deliberately. Vary speed, size, and direction randomly within each recording. Avoid settling into a rhythm that looks more like fast FLOW.

Common mistake Rhythmic fast motion (like a steady drumbeat). This can be too predictable for CHAOS and may be classified as FLOW by the trained model.

9. Training Procedure — Step by Step

9.1 Pre-Training Checklist

Before opening Wekinator, verify:

1. Both Processing sketches are closed.
2. The webcam is plugged in and recognized by the operating system.
3. The room is reasonably well lit. The motion detector is sensitive to lighting changes, so unstable lighting will produce noisy training data.
4. No other application is using ports 6448, 12000, or 12001.
5. The oscP5 and netP5 libraries are installed in the Processing library manager.
6. The Processing video library is installed and at least one camera is returned by `Capture.list()`.

9.2 Launch Sequence

1. Open Wekinator and create a new project.
2. Configure: 30 inputs, 1 output, type = Classification.
3. Run `VideoMotionDetection.pde`. Confirm the sketch window appears and the HUD is visible.
4. Watch the **OSC In** indicator in Wekinator. It must turn green within one second. If it does not, recheck the port configuration.
5. Run `CameraTint.pde`. Confirm the second camera window appears.
6. The **OSC Out** indicator in Wekinator will not turn green until after training and Run are initiated.

9.3 Recording Class 1: STILL

1. Set the output label to **1** in Wekinator.
2. Position yourself in front of the camera.
3. Watch the buffer waveform for 5–10 seconds. Confirm values are near zero.
4. Click **Record** in Wekinator.
5. Hold completely still for 4–6 seconds.
6. Click **Stop**.
7. Repeat 4 more times, varying your position slightly each time.

Training tip for STILL

Aim for 15–20 total examples for robust STILL classification. If the waveform shows values above 0.10 during a STILL recording, eliminate the background movement before continuing.

9.4 Recording Class 2: FLOW

1. Change the output label to **2** in Wekinator.
2. Begin sustained smooth motion and watch the waveform until it shows a consistent mid-level pattern.
3. Click **Record**.
4. Continue the smooth motion for 5–8 seconds without pausing.
5. Click **Stop**.
6. Rest briefly, then perform a different type of smooth motion and repeat 4 more times.

Training tip for FLOW

Variety matters most for FLOW. Include slow arm waves, medium body sways, and slow walks to give the model diverse examples of the continuity quality.

9.5 Recording Class 3: CHAOS

1. Change the output label to **3** in Wekinator.
2. Begin vigorous, irregular motion. Confirm waveform spikes high and varies rapidly.
3. Click **Record**.
4. Maintain intense, irregular motion for 5–8 seconds.
5. Click **Stop**.
6. Rest, then perform a different type of chaotic motion and repeat 4 more times.

Training tip for CHAOS

Vary between fast hand shaking, large body movements, and sudden direction changes. Monotonous fast motion can be confused with FLOW. Unpredictability is the defining feature of CHAOS.

9.6 Training and Running the Model

1. Verify example counts show at least 5 per class (10+ preferred).
2. Click **Train** in Wekinator. With kNN, this is nearly instantaneous.
3. Click **Run**. The **OSC Out** indicator should now turn green.
4. Move in front of the camera. Test each class in turn.
5. If results are poor, go back and record more examples. Focus on the class being confused with another. Make demonstrations more extreme and distinct.

9.7 Iterative Improvement

Interactive machine learning is inherently iterative. Use this loop:

1. Train and run the initial model.
2. Test each class and note which is most frequently misclassified.
3. Identify what the misclassified motion looks like on the waveform.
4. Record additional examples of the confused class, making them more extreme and distinct.
5. Click Train again in Wekinator (new examples are added to existing ones).
6. Test again and repeat until satisfied.

In practice, 3–5 iterations are sufficient to produce a reliable classifier.

10. OSC Communication — Complete Reference

Open Sound Control (OSC) is a network protocol designed for real-time communication between creative applications. It transmits typed arguments (floats, integers, strings) addressed by a URL-like pattern string.

10.1 OSC Message Format

Direction	Address	Arguments	Destination	Rate
Sketch → Wekinator	`/wek/inputs`	30 floats	`:6448`	≈30/s
Wekinator → Sketch	`/wek/outputs`	1 float	`:12000`	≈30/s

10.2 Port Map

Process	Listens on	Sends to	Protocol
VideoMotionDetection	port 12001	port 6448 (Wekinator)	OSC/UDP
Wekinator	port 6448	port 12000 (CameraTint)	OSC/UDP
CameraTint	port 12000	— (output only)	OSC/UDP

Why does VideoMotionDetection listen on 12001? oscP5 requires a listening port to initialize even when only sending. Port 12001 is chosen to avoid conflict with CameraTint (12000). No messages are expected on 12001 in normal operation.

10.3 Debugging OSC Connectivity

1. Confirm VideoMotionDetection is running and the HUD shows the correct host and port.
2. Check the **OSC In** indicator in Wekinator. If not green, VideoMotionDetection is not sending or Wekinator is not listening on the right port.
3. Check the Processing console for oscP5 initialization messages. Any binding errors appear there.
4. Use a tool such as Protokol (free) to sniff OSC traffic on port 6448 and confirm messages are arriving with exactly 30 float arguments.
5. If **OSC Out** in Wekinator never turns green, confirm CameraTint is running *and* that the model has been trained and Run has been clicked.

11. Improving Classification Quality

The quality of a kNN classifier is almost entirely determined by the quality of its training data. There are no hyperparameters to optimize and no training loss to minimize — the model memorizes examples. Every improvement must come from better demonstrations.

11.1 Data Quality Principles

- **Class Separation.** The three classes must be behaviourally distinct in the waveform before recording begins. If your FLOW sometimes looks like CHAOS because you moved too fast, the model will be confused in exactly those situations.
- **Coverage.** Each class should be demonstrated in a variety of ways. STILL recorded only with arms at the sides may fail when you are still with arms raised. Record STILL in different postures, FLOW with different body parts and speeds, CHAOS with different types of irregular motion.
- **Environmental Consistency.** Record training examples in the same lighting conditions you will test in. Daylight shifting to artificial light changes the brightness distribution of pixels and can shift classification boundaries.
- **Balance.** Aim for roughly equal example counts across all three classes. A large imbalance biases kNN toward the over-represented class.

11.2 Parameter Tuning

Parameter When and how to adjust
threshold = 30 Increase to 50 if background noise causes STILL to show unexpected motion. Decrease
bufferSize = 30 Increase to 60 for more temporal context. Decrease to 15 for more responsiveness. N
multiplier ×4 Increase to ×6 if motionAmount rarely exceeds 0.3 during vigorous motion. Decrease
lerp factor 0.1 Increase for snappier tint response. Decrease for smoother but slower transitions.

12. Educational Significance

This project is a complete pedagogical example of interactive machine learning within a single coherent, self-contained system. Every major concept in the field is represented:

Concept	How it appears in this project
Feature Engineering	The system does not learn from raw pixels. It learns from a hand-designed temporal buffer. The quality of this feature is the upper bound on classification quality.
Supervised Learning	Classes are defined by the user through demonstration. The model only knows what STILL means because a human showed it and labeled those examples.
Generalization	After training, the model classifies motion it has never seen before by finding similar examples in its training set.
Real-Time Feedback	The closed loop between movement, sensing, classification, and visual output creates a system where the user observes model behavior immediately.
Iterative Development	The workflow of demonstrating, training, testing, and revising mirrors professional machine learning practice at comprehensible scale.
Human-in-the-Loop	The meaning of STILL, FLOW, and CHAOS is entirely determined by the human. The machine learns the boundaries; the semantics come from the person.

12.1 Limitations

- **Single-channel perception.** The system cannot distinguish two people moving identically in different parts of the frame, or fast movement of a small object from slow movement of a large one.
- **Session persistence.** Wekinator training data is saved with the project file. The model must be retrained if Wekinator is relaunched without loading the saved project.
- **Camera dependence.** Different cameras have different noise profiles and compression artifacts. A model trained on one camera may not transfer to a different camera without retraining.

12.2 Possible Extensions

- Add a fourth class (e.g., PULSE — rhythmic, regular motion like clapping) to probe the boundaries of the classifier.
- Replace the single-channel brightness diff with separate motion values for the top, middle, and bottom thirds of the frame, increasing input dimensionality to 90 (while updating Wekinator accordingly).
- Connect the predicted class to an audio synthesizer to build a motion-controlled instrument.
- Use Wekinator regression output instead of classification to control a continuous parameter such as reverb depth or animation speed.
- Experiment with optical flow algorithms for richer motion descriptors.

13. Quick Reference Card

Startup Sequence

1. Open Wekinator — configure: 30 inputs, 1 output, Classification
2. Run `VideoMotionDetection.pde` — confirm OSC In turns green in Wekinator
3. Run `CameraTint.pde` — confirm second camera window appears
4. Record 5+ examples for Class 1 (STILL)
5. Record 5+ examples for Class 2 (FLOW)
6. Record 5+ examples for Class 3 (CHAOS)
7. Click **Train** in Wekinator
8. Click **Run** in Wekinator — confirm OSC Out turns green
9. Move in front of camera and observe the tint change

Troubleshooting

OSC In not green	Wrong port in Wekinator, or VideoMotionDetection not running
OSC Out not green	CameraTint not running, or model not yet in Run mode
Tint never changes	Check `/wek/outputs` address; check typetag length
All predictions = 1	Too few or identical examples. Record more diverse data.
FLOW misclassified	Speed too fast ($\rightarrow$ CHAOS) or too slow ($\rightarrow$ STILL)
CHAOS not detected	Motion not irregular enough. Make it more unpredictable.
Buffer waveform flat	Threshold too high, or lighting too low. Try threshold 15.

Key Code Parameters to Tune

File	Parameter	Default
VideoMotionDetection	`threshold`	30
VideoMotionDetection	`bufferSize`	30 (must match Wekinator)
VideoMotionDetection	`mirror`	`true`
VideoMotionDetection	amplifier	$\times 4$
CameraTint	lerp factor	0.1
CameraTint	overlay alpha	80
Wekinator	k (kNN)	3

Real-Time Motion Classification with Wekinator and Processing
End of Report